



Deep reinforcement learning based computation offloading and resource allocation strategy for maritime internet of things

Yanli Xu , Qianlian Yu

[Show more](#)

Outline | Share Cite

<https://doi.org/10.1016/j.comnet.2025.111221>

[Get rights and content](#)

[Full text access](#)

Abstract

With the continuous growth of marine activities, the demands of computation-intensive applications dramatically increase. How to provide guaranteed computation service for maritime Internet of Things (M-IoT) is not well addressed. In this paper, we make use of space-air-ground-sea integrated network (SAGSIN) and mobile edge computing (MEC) to provide wide-coverage and efficient computation services for M-IoT users. The heterogeneous network resources of SAGSIN are fully used by unified management based on software-defined networking (SDN). In addition, we propose a joint computation offloading and resource allocation algorithm based on dueling double deep Q network (D3QN) with the consideration of heterogeneous SAGSIN architecture and scarce service resources of marine scenarios. The proposed algorithm can minimize delays and energy consumption by optimizing task offloading decisions and resource allocation of communication and computation. It achieves efficient resource utilization and ensures that the processing power is highly matched with the task requirements. The simulation results demonstrate that the proposed algorithm outperforms Deep Q Network (DQN), Double DQN, and Dueling DQN in terms of the overall system overhead. Furthermore, the proposed algorithm exhibits exceptional performance and robustness. It can ensure the increasing demand for computing services in Marine networks.

Previous

Next

Keywords

Maritime Internet of Things (M-IoT); Mobile edge computing (MEC); Computation offloading; Resource allocation; Deep reinforcement learning

1. Introduction

In recent years, maritime activities such as marine tourism, offshore aquaculture, and oceanic mineral exploration have seen rapid development [1,2], which has greatly promoted the development of the maritime Internet of Things (M-IoT). With the increasing number of vessels, offshore platforms, buoys, etc., there has been a growing demand for high-speed and ultrareliable maritime communications to connect them. Due to the complexity of maritime geography and meteorological environment, maritime wireless communication channels have unique propagation characteristics that are different from land, such as sparsity and instability, which leads to sparse network communication links and complex task offloading [3]. In maritime networks, ships sail and the communication coverage is limited, which makes it difficult to guarantee reliable maritime communication services. In addition, when the edge network faces a large number of computing tasks, M-IoT devices are limited by hardware and cannot carry the computing and transmission of a large amount of data. However, communication and computing resources that rely solely on terrestrial networks to support maritime services are often limited and have been unable to meet the growing demand for maritime network access. Based on the above challenges, it is imperative to develop more efficient network and computing solutions to achieve better performance in massive maritime data transmission and task processing.

Space-air-ground-sea integrated network (SAGSIN) provides seamless network services to space, aerial, ground, and maritime users, satisfying the demands on all-time all-area coverage of wireless communications. It covers space-based networks, air-based networks, sea-based networks and traditional terrestrial networks. This integrated network design helps overcome the limitations of a single network type in terms of coverage and network capacity. However, the research on SAGSIN is still in the preliminary exploration stage, especially in maritime service applications. The existing research results are mainly accumulated in the integration of each network segment, or two or three network segments, such as space and ground network, air and ground network, air and sea network, and space-air-ground integrated network [[4], [5], [6], [7]]. For example, in order to satisfy the quality of service (QoS) requirement of marine applications, a hybrid offshore and aerial-based multi-access mobile edge computing (MEC) framework in [4] was proposed to minimize the maximum workload delay. In [5], a hybrid satellite-UAV-terrestrial maritime communication network was investigated, in which satellites and terrestrial systems were integrated for maritime coverage enhancement. A collaborative space/aerial-aided MEC framework for 6 G system in [6] was proposed in which the satellites are able to offload the tasks to nearby aircrafts through one-hop connection or cloud server along multi-hop offloading path. Compared to terrestrial networks, the framework has a higher offloading efficiency when dealing with large-scale tasks.

To achieve seamless and comprehensive coverage of the ocean, a novel space-air-sea (SAS)- non-terrestrial networks (NTNs) architecture was proposed in [8] that enabled each maritime UE to connect with the terrestrial networks, in which the NTNs integrate space and aerial networks with terrestrial systems. A space-air-ground-sea integrated network architecture with edge and cloud computing introduced in [9], was designed to provide flexible hybrid computing services for marine users, which improves the communication and computing efficiency greatly. Considering the optimization of resources and the collaboration of heterogeneous networks, a prevailing trend involves the integration of soft-defined network (SDN) into the design of SAGSIN. SDN separates the control plane and data plane from the control plane, and the control plane centrally controls the network equipment through the open interface [10]. For example, a software-defined air-ground integrated vehicle network was proposed to achieve flexible, reliable and scalable network resource management in [11]. In [12], to solve the problem of low efficiency of network management, the method combining SAGIN with SDN was proposed to simplify the complexity of the network and improve network performance. The above research has made significant contributions to promoting network convergence, especially in the integration of multiple network segments, further promoting seamless and comprehensive coverage of the ocean and simplifying the complexity of network management.

Regarding the problem of high transmission delay and low bandwidth in the conventional cloud computing approach, the mobile edge computing is a new revelation that extends cloud computing services such as data computation and resource storage capabilities to the edge of the maritime network [13,14]. M-IoT devices can offload computing tasks to edge servers in the network to reduce the adverse impact of resource constraints such

as computing, memory, and energy. Many researchers have proposed their own schemes to improve task offloading efficiency. For example, a voyage-based computation offloading mechanism was proposed for compute-intensive applications at sea in [15]. Shipborne base stations can dynamically provide mobile edge computing services for nearby users, which effectively improve the delay, energy consumption, and traffic cost of the offloading tasks in maritime edge networks. Similarly, with energy consumption and delay constraints, a task offloading scheme based on the ship motion model was proposed in [16]. The proposed scheme can improve the successful rate of task execution and reduce network packet loss rate while ensuring resource load balancing and the stability of the network. In [17], a multi-vessel computing offloading algorithm based on improved Hungarian algorithm was proposed to solve the offloading problem of multi-ship computing tasks in maritime mobile edge computing networks. In order to reduce the burden of ship IoT users from compute-intensive tasks, the problem of selecting optimal edge server under the multi-armed bandits (MAB) framework environment was studied in [18]. The proposed algorithm can achieve considerably lower offloading latency and weighted latency-energy cost. In addition, a double-edge secure offloading scheme was proposed to minimize offloading delay under the constraint of security requirements in [19], where maritime mobile users can adaptively offload their computation tasks to the BS or satellites securely relayed by unmanned aerial vehicles (UAVs).

From the perspective of overcoming disadvantages brought about scarce communication resources of M-IoT, the academic community has proposed a variety of advanced technologies and algorithms. In [20], the authors addressed a dynamic offloading problem involving energy-delay trade-off and proposed a two-stage joint optimal offloading algorithm. This algorithm aims to optimize the allocation of computation and communication resources for users under constraints of limited energy and sensitive latency. The authors in [21] proposed to utilize the UAV as a computing node and relay node to improve the average user delay in the UAV-aided MEC (UAV-MEC) network and minimized the average latency of all users by jointly optimizing UAV placement, UE association, and communication resource assignment. The authors in [22] presented the sum power minimization problem via jointly optimizing user association, power control, computation capacity allocation, and location planning in a UAV-enabled MEC network. The authors in [23] developed a joint computation offloading and resource allocation algorithm based on the Lyapunov optimization technique to maximize the long-term average throughput by optimizing task offloading, subchannel allocation, computing resource allocation, and task migration decisions. By offloading compute-intensive tasks to edge servers, the computing burden on devices can be reduced and the delay of task processing can be lowered, thereby improving the user experience. However, computational complexity can lead to reduced operability in real-world applications, especially at resource-constrained edge environments.

The deep reinforcement learning (DRL) method explores dynamic unknown environments to make computation on offloading and resource allocation, in order to solve the joint optimization problems with high complexity in marine environments [[24], [25], [26], [27]]. For instance, the authors in [24] focused on the decision of maritime task offloading by the cooperation of UAVs and vessels, and proposed a Q-learning based approach to minimize the total execution time and energy cost. To improve the execution efficiency of maritime edge computing under the constraints of delay and energy consumption, a task offloading algorithm based on reinforcement learning by means of cloud-edge-end cooperation was proposed in [25]. A cooperative multi-agent deep reinforcement learning algorithm in [26] was investigated to address the task offloading problem by jointly designing the trajectories, computation task allocation, and communication resource management of UAVs. The authors in [27] focused on optimizing offloaded workload, transmit power, computing resource allocation of the unmanned surface vehicle (USV) and the UAV trajectory to minimize the total energy consumption of the UAV and USV, so the authors proposed a two-layered algorithm, where the top-layer algorithm based on deep reinforcement learning deep deterministic policy gradient (DDPG) and the bottom-layer algorithm based on Lagrange multiplier method was used to obtain the optimal solution.

Most of existing studies target specific scenarios and lack scalability. The complexity of the algorithm makes it difficult to meet the real-time requirements, which is especially prominent in the dynamically changing ocean

scene. Therefore, more flexible and adaptive task offloading and resource allocation algorithms need to be designed for different scenarios and diverse operating conditions.

In this paper, we investigated the issue of joint computation and communication resource management in space-air-ground-sea integrated maritime network, focusing on minimizing the overall total cost of computation tasks. A task offloading and resource allocation algorithm based on DRL was proposed to optimize binary offloading decisions, computing resource allocation and power allocation. The main contributions of this paper are as follows.

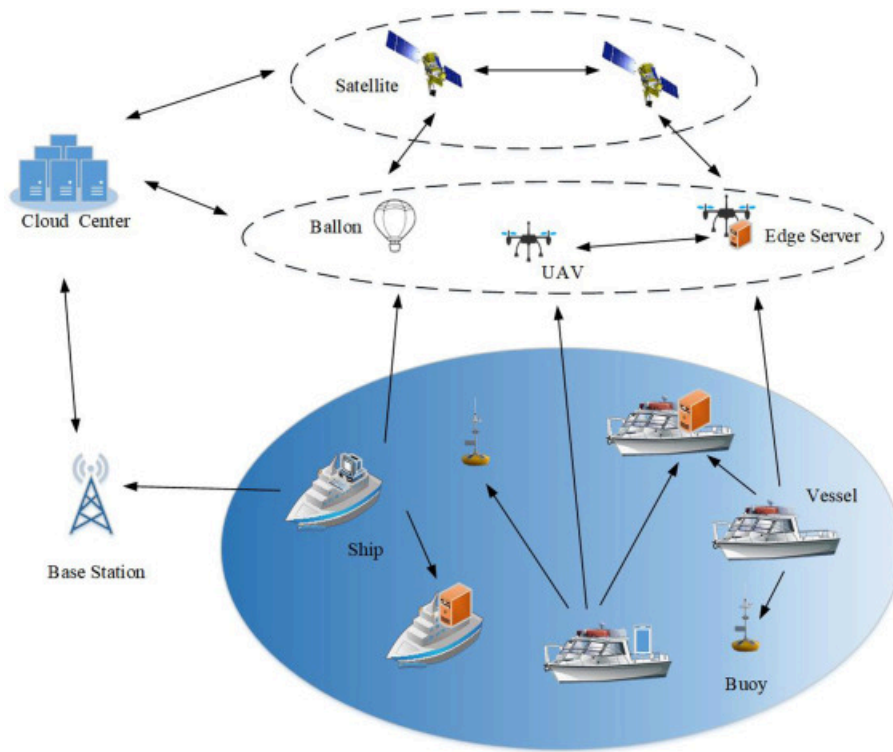
- 1) We introduce the integrated network architecture of space-air-ground-sea based on SDN to provide flexible hybrid computing services for maritime networks. In order to make full use of the scarce communication and computing resources for the heterogeneity of SAGSIN, we use SDN technology to achieve unified management and coordinated control of marine network resources.
- 2) In order to minimize the total delay and energy consumption of task offloading, we design a task offloading and resource allocation algorithm based on Dueling Double Deep Q network (D3QN) to jointly optimize task offloading strategy, transmission power and computing resources, which overcomes shortcomings of over-fitting of Dueling Deep Q Network (DQN) algorithm and inaccuracy of Double DQN algorithm.
- 3) We design simulation experiments to prove that the D3QN algorithm is obviously better than other basic algorithms in reducing the total cost of the system, and we test the algorithm under different system model parameters to evaluate the performance of the proposed algorithm. Simulation results show that the proposed D3QN algorithm has better stability and faster convergence speed, which confirms its applicability in the M-IoT scenario.

The rest of this paper is organized as follows. The system model and problem formulation are described in [Section 2](#). In [Section 3](#), we proposed a computation offloading and resource allocation scheme. To verify the effectiveness of the proposed algorithm in [Section 4](#), we conduct numerical simulation and analysis. Finally, the conclusion is provided in [Section 5](#).

2. System model and problem formulation

2.1. network model

We propose a heterogeneous SAGSIN composed of M-IoT devices (ships, buoys and ocean monitoring stations, etc.), UAVs, base stations and satellites, as shown in [Fig. 1](#).



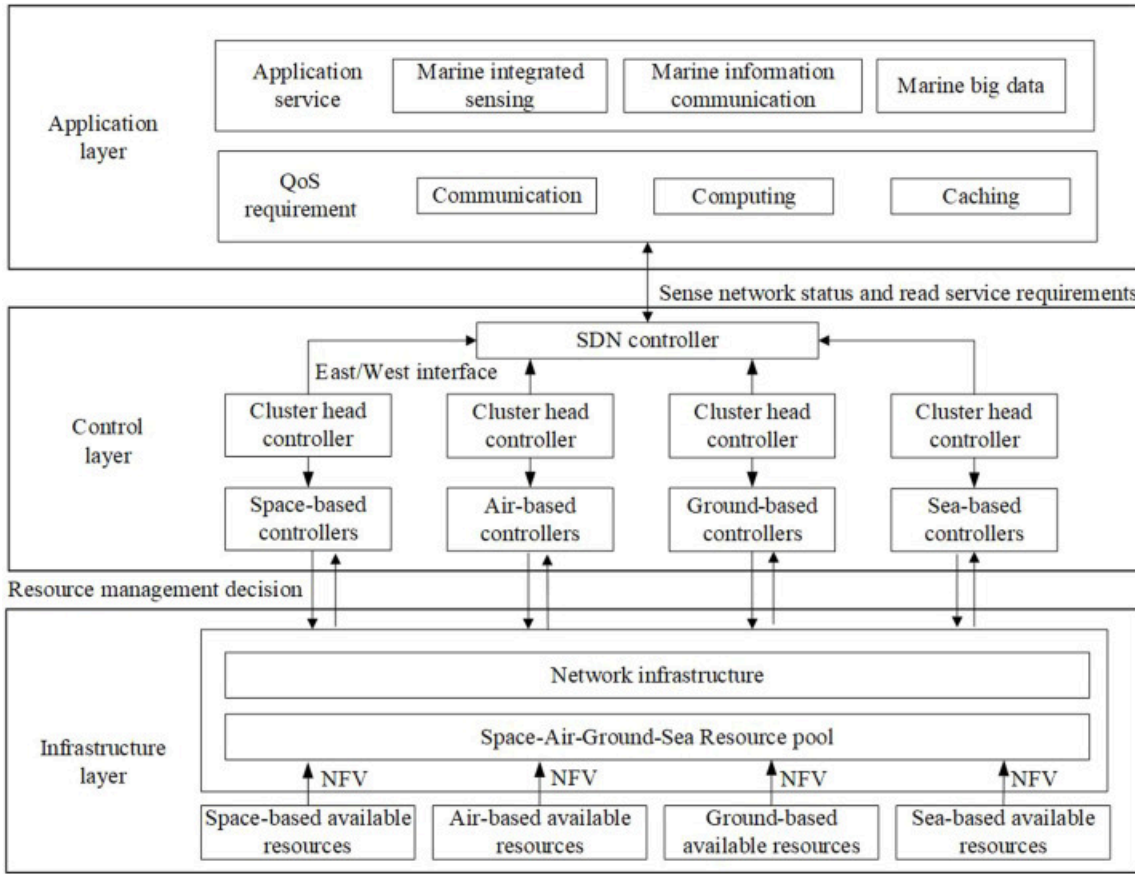
[Download: Download high-res image \(446KB\)](#)

[Download: Download full-size image](#)

Fig. 1. space–air–ground–sea integrated network.

The communication resources are located in low earth orbit (LEO) satellites and UAVs to provide wide-area coverage and efficient communication connectivity, and the computing resources are distributed in edge computing servers for M-IoT devices. Let $\mathcal{N} = \{1, 2, \dots, N\}$ represents the set of M-IoT devices in a specific region, where each device can generate compute tasks independently and optionally compute them locally or offload them to a MEC server. And $\mathcal{M} = \{1, 2, \dots, M\}$ is the set of MEC servers that are deployed on UAVs, offshore platforms or ships to support remote processing of computing tasks. Applications and related services run by M-IoT devices will generate computing tasks. The computation tasks generated by each M-IoT device can be expressed as $W_i = (B_i, C_i, \tau_i^{\max})$, where B_i represents the size of the task calculation input data, C_i represents the computing CPU cycles required to complete the computing task W_i , $\tau_i^{\max}$ represents the maximum tolerable delay of the task. Considering the transmission distance and bandwidth limitations of ocean communications, coupled with the relatively insufficient computing power of M-IoT devices to deal with complex tasks, the computing tasks should be offloaded to the MEC server [28]. MEC servers are equipped with computing hardware and software frameworks, including computing nodes, storage systems, and network interfaces, and are capable of providing computing services, including data processing, analysis, and storage, to M-IoT devices.

In order to effectively manage heterogeneous communication and computing resources, SDN technology can separate network management and data distribution functions and provide flexible and dynamic resource management [11] as shown in Fig. 2. The architecture of SAGSIN consists of three parts, i.e., infrastructure layer, control layer and application layer (Fig. 2). At the infrastructure layer, network Functions Virtualization (NFV) is used to virtualize physical resources from different network segments. Infrastructures mainly provide communication resources and computing resources. The control layer is mainly responsible for flexibly allocation of the LEO/UAV communication channels and the computation resource in MEC servers for vessels which request communication and computation resource. The SDN controller is responsible for centralized management and control of network resources to achieve flexible and efficient data flow control and network configuration. Communication and computing decisions are made based on scarce communication resources and the status of the MEC server. The application layer mainly provides various types of maritime applications, such as ocean information detection, ocean comprehensive perception.



[Download: Download high-res image \(767KB\)](#)

[Download: Download full-size image](#)

Fig. 2. SDN-based SAGSIN architecture.

2.2. Communication model

Due to the uniqueness of the maritime propagation environment, the modeling of ocean channels is different from that of terrestrial networks. In maritime communications, transmission failures often occur frequently due to dynamic changes in channel conditions. We use a two-ray signal propagation model here [29]. Assuming that both the transmitting antenna and the receiving antenna have unit gain, the wireless channel gain between the M-IoT i device and the edge server MEC j can be expressed as:

$$h_{ij} = \left(\frac{\lambda}{4\pi d}\right)^2 4\sin^2\left(\frac{2\pi h_T h_R}{\lambda d}\right) \quad (1)$$

where h_T and h_R are the transmitter and receiving antenna heights (m), respectively, and λ is the carrier wavelength (m), d is the communication distance (m).

Each M-IoT device can offload the computation task to the deployed MEC server when the task is hard to be executed locally under restrained conditions. Assuming that the total communication duration is divided into T time slots, and each time slot is denoted as t . If frequency division multiple access (FDMA) is used between M-IoT devices and MEC servers [30], then in time slot t , the transmission rate between M-IoT i and MEC j can be expressed as

$$R_{ij} = B \log_2 \left(1 + \frac{p_i h_{ij}}{\sigma^2}\right) \quad (2)$$

where B denotes the channel bandwidth of the available spectrum (MHz), p_i is the uplink transmit power of M-IoT i (dBm), and σ^2 is the Gaussian white noise power (dBm).

2.3. Computation model

For each ship user, its tasks can be executed on the local CPU or offloaded to the MEC server. Achieving efficient computation offloading and resource allocation in MEC-assisted maritime networks remains a challenge. In order to simplify the problem, the user terminal in the MEC system studied in this article considers using binary offloading. Therefore, the offloading strategy of M-IoT devices is $\alpha_{ij} = \{0, 1\}$, which represents the set of offloading decisions of all user equipment, where $\alpha_{ij} = 0$ means that the task is executed locally, and $\alpha_{ij} = 1$ means that the task is processed in the edge server.

1) Local Computing Model

When ship user i chooses to perform a computing task locally, the calculation process is only related to the ship user's CPU computation power. We denote f_i^l as the local CPU computation power of ship user i , i.e., the number of CPU cycles per second. The delay required for local calculation of task, which can be expressed as

$$T_i^l = \frac{C_i}{f_i^l} \quad (3)$$

Correspondingly, the energy consumption required by ship user i to perform computation tasks locally, which can be defined as

$$E_i^l = \varepsilon_i C_i \quad (4)$$

where ε_i is the energy consumption coefficient (J/cycle), which represents the energy consumed by each CPU cycle of user i 's local device. It is a super linear function of the user's CPU computing power. The energy consumption model is set as $\varepsilon_i = k(f_i^l)^2$, where k is the energy consumption coefficient of the chip (J/s), which depends on the chip of the device structure, taken here $k = 10^{-27}$.

Therefore, we calculate the total cost of executing the task locally as:

$$C_i^l = \lambda_{loc} T_i^l (1 - \lambda_{loc}) E_i^l \quad (5)$$

where λ_{loc} is the weight parameter of the local computing delay, $\lambda_{loc} \in [0, 1]$.

2) Offloading Computation Model

Multiple M-IoT devices can be associated with the same MEC server and offload tasks to the MEC server at the same time, so that each user is only allocated a portion of the computing resources. We denote the total computing resources of each MEC server with $\sum_{i \in \mathcal{N}} \alpha_{ij} f_{ij}^o \leq F$.

The task completion delay mainly consists of three parts. The first one is the uplink wireless transmission delay for offloading task W_i from device i to MEC j . The second one is task computing delay, which is related to the allocated computing resources f_{ij}^o . The third part is the computation result of the downlink wireless transmission delay from MEC j to device i . Since the data size after task processing is much smaller than the input data size, and the downlink transmission rate is much higher than the uplink transmission rate, we ignore the delay and energy consumption of the download transmission [9]. When ship user i decides to offload the computing task to MEC j , the task completion delay can be written as

$$T_{ij}^o = \frac{B_i}{R_{ij}} + \frac{C_i}{f_{ij}^o} \quad (6)$$

The total energy consumption for completing task W_i is given by

$$E_{ij}^o = \frac{p_i B_i}{R_{ij}} + k(f_{ij}^o)^2 C_i \quad (7)$$

where p_i is the transmission power of user i .

Therefore, we calculate the total cost of executing the task in MEC server as

$$C_{ij}^o = \lambda_{mec} T_{ij}^o + (1 - \lambda_{mec}) E_{ij}^o \quad (8)$$

where λ_{mec} is the weight parameter of delay in edge computing, $\lambda_{mec} \in [0, 1]$.

2.4. Problem formulation

In this subsection, we address the computational offloading optimization problem for multiple ship user equipment. In the MEC system, the total cost of the system is mainly determined by the delay and energy consumption of completing the task. Combined with the previous analysis, the minimization problem of the weighted sum cost for all M-IoT devices and MECs in the network can be expressed as

$$C_{total}(\alpha_{ij}, p_i, f_{ij}^o) = \sum_{i \in \mathcal{N}} \sum_{j \in \mathcal{M}} (1 - \alpha_{ij}) C_i^l + \alpha_{ij} C_{ij}^o \quad (9)$$

In order to improve the user's task offloading efficiency, this paper mainly minimizes system costs by optimizing offloading decisions α , transmission power p and computing resource allocation f . The optimization problem of joint computing offloading decision-making and resource allocation can be formulated as.

$$\begin{aligned} P1 : \min_{\alpha, p, f} C_{total}(\alpha_{ij}, p_i, f_{ij}^o) \quad s. t. \quad & C1 : \alpha_{ij} \in \{0, 1\} \quad C2 : \lambda_{loc}, \lambda_{mec} \in [0, 1] \\ & C3 : \sum_{i \in \mathcal{N}} \alpha_{ij} f_{ij}^o \leq F \quad C4 : 0 \leq f_{ij}^o \leq F \quad C5 : 0 \leq p_i \leq P_{max} \end{aligned} \quad (10)$$

Constraint C1 ensures that the computing offloading decision is a binary variable; constraint C2 is a trade-off parameter based on user needs; constraint C3 ensures that the total amount of allocated computing resources does not exceed the total computing resources of the MEC server; constraint C4 indicates; that the amount of computing resources allocated to M-IoT devices does not exceed the maximum computing resources of the MEC; constraint C5 ensures that the ship user's transmit power does not exceed the maximum transmit power.

The optimization problem in (10), the optimization task involves jointly designing the transmission power of devices, computation offloading modes, and resource allocation strategies to maximize system performance, specifically by minimizing delay and energy consumption. Due to the optimization parameters comprising both binary discrete variables and continuous variables, this problem is classified as a mixed-integer non-convex optimization problem, categorizing it as NP-hard and rendering traditional mathematical methods inefficient for its resolution. To effectively address this optimization challenge, we develop a deep reinforcement learning-based algorithm capable of autonomously learning and optimizing decision-making strategies in high-dimensional state spaces. This approach enables efficient resolution of the complex joint optimization problem, significantly enhancing system performance in dynamic and resource-constrained environments.

3. DRL-based computation offloading and resource allocation schemes

In section II, we formulated the joint optimization problem of computation offloading and resource allocation with the objective of optimizing the delay and system energy consumption of the computation tasks. To cope with the computational complexity due to the complexity of the scenario, in this section, we propose a scheme for computation offloading and resource allocation based on the D3QN algorithm, which learns the action value function for each action in all states and selects the action with the maximum cumulative reward discount to obtain the optimal offloading and allocation policy.

3.1. Markov decision process formulation

Due to the optimization parameter is a combination of the binary discrete variables α and continuous variables p and f , the problem (10) can be regarded as a mixed-integer and nonconvex problem, which is NP-hard and difficult to solve by traditional mathematical methods. Therefore, we propose a Deep Reinforcement Learning (DRL) based algorithm to address this problem. In order to apply DRL, we firstly transform the problem into

Markov decision process (MDP), which contains four critical factors: agent, state, action, reward. We denote SDN controller as an agent to learn the optimal strategy. Then, we define state space, action space, reward function and policy as follow.

1) State Space: The state at time slot t , includes the available computing resources of MEC, the data size of the current computing task, and the channel status information between the ship user and the MEC. The state at time slot t can be defined as

$$\mathbf{s}_t = \{\mathbf{h}_{ij}(t), \mathbf{B}_i(t), \mathbf{f}_j(t)\} \quad (11)$$

where $\mathbf{h}_{ij}(t) = [h_{11}(t), \dots, h_{1M}(t), \dots, h_{N1}(t), \dots, h_{NM}(t)]$ represents the channel gain between the M-LoT i and the MEC j at the time slot t ; $\mathbf{f}_j(t) = [f_1(t), \dots, f_M(t)]$ denotes the available computing resources of MEC; $\mathbf{B}_i(t) = [B_1(t), \dots, B_M(t)]$ represents the data size of the current computing task.

2) Action: The joint computation offloading and resource allocation problem consists of three components: offloading decision-making and transmission power allocation of the M-LoT devices, computing resource allocation of MEC servers. Due to the actions of offloading decision are discrete, but the actions of transmission power allocation and heterogeneous computing resource allocation are continuous, we integrate the action refinement into DRL, so action at time slot t can be defined as

$$\mathbf{a}_t = \{\boldsymbol{\alpha}(t), \mathbf{p}(t), \mathbf{f}(t)\} \quad (12)$$

where $\boldsymbol{\alpha}(t) = [\alpha_{11}(t), \dots, \alpha_{1M}(t), \dots, \alpha_{N1}(t), \dots, \alpha_{NM}(t)]$ represents offloading decision about whether or not to offload tasks to MEC servers; i.e., if $\alpha_{ij}(t) = 1$, task W_i will be offloaded to MEC j , otherwise the task is executed locally; $\mathbf{p}(t) = [p_1(t), \dots, p_N(t)]$ denotes the transmission power of each ship user at the time slot t ; $\mathbf{f}(t) = [f_{11}^o, \dots, f_{1M}^o, \dots, f_{N1}^o, \dots, f_{NM}^o]$ represents the computing resource that each MEC server assigns to each device at time slot t .

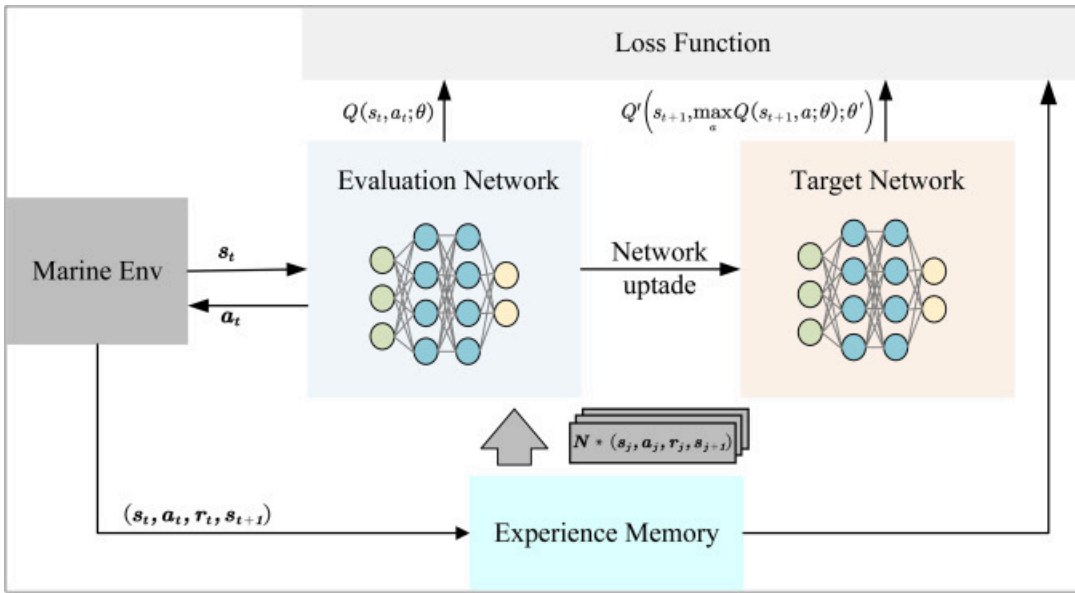
3) Reward: In reinforcement learning, a reward is a scalar signal that the environment provides to the agent following the execution of an action in a specific state, thereby facilitating the improvement of the agent's policy. While the objective of reinforcement learning is to maximize the system's long-term average reward, the objective of Problem P1 is to minimize the system's total cost. To integrate the reward function with the optimization objective, this study defines the reward as the negative of the system's total cost. By doing so, maximizing the reward effectively corresponds to minimizing the system cost, thereby aligning the reinforcement learning framework with the underlying optimization goals. The reward function can be defined as

$$r_t(\mathbf{s}_t, \mathbf{a}_t) = -\mathbb{E} \left[\sum_{i \in \mathcal{N}} \sum_{j \in \mathcal{M}} (1 - \omega) C_i^l + \omega C_{ij}^o \right] \quad (13)$$

where C_i^l is the total cost of performing the task locally, C_{ij}^o is the total cost of performing the task at the MEC server, and ω is the weighting factor.

3.2. D3QN-Based computation offloading and resource allocation algorithm

In our proposed network, the number of states and actions will increase with the increase of the number of M-LoT devices and MECs, and the corresponding computational complexity will increase exponentially. To address this problem, in our design, we discretize the action space A while keeping the state space S continuous. If both state and action spaces are discretized at the same time, it would lead to poor performance due to a large number of state-action pairs [31]. Therefore, we propose a computation offloading and resource allocation algorithm based on Dueling Double DQN (D3QN), which learns the action value function for each action under all states and selects the action with the maximized cumulative reward discount to obtain an optimal strategy. The network architecture is shown in Fig. 3.



Download: [Download high-res image \(330KB\)](#)

Download: [Download full-size image](#)

Fig. 3. The network structure of D3QN.

In each time step t , the agent first observes a state s_t based on the environment and puts it into a DNN. The input of the D3QN network is the current state s_t and the output is the Q value of each possible action a_t at the state s_t , which can be presented as $Q(s_t, a_t)$. The agent selects an action according to the ϵ -greedy policy then perform the action, which can make the agent explores the unknown action and state in each step so as to avoid the algorithm falling into a locally optimal solution. After executing the corresponding action, the agent transfers to the next state and gets the reward r_t . Finally, experiences (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer for training of the network [32].

During the training process, the object of the D3QN network training is to obtain a series of actions that can achieve the maximized accumulated discounted reward. The D3QN algorithm splits the output $Q(s_t, a_t)$ into two different parts, which is the state value function $V(s; \theta, \beta)$ and action advantage function $A(s, a; \theta, \alpha)$ individually expressed as

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha) \quad (14)$$

where θ represents the neural network parameter, β and α are the network parameters of the state value function and the action advantage function, respectively.

To achieve a better performance of the network training, above formula can be reformulated as

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left[A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha) \right] \quad (15)$$

where $\frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)$ represents the average value of all possible action advantage functions in the current state, and $|\mathcal{A}|$ represents the number of selectable actions.

Then, we adopt fixed Q-target technology to improve the stability and convergence of training. We construct two neural networks with the same structure but different parameters. One is called the evaluation Q-network which represents Q value and is denoted as $Q(s_t, a_t, \theta)$, where θ is the weights of evaluation Q-network. Another is called the target Q-network which is used to calculate the $Q_{\text{target}} y_t$. Specifically, the evaluation network selects the action corresponding to the maximum Q value in the current Q network and then places the selected action with the maximum value in the target network to calculate the target Q value. Under this case, the update

formula for the target Q-value is written as

$$y_t = r_t + \gamma \max_{a'} Q' \left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta' \right) \quad (16)$$

where θ' is the weights of target Q-network. The evaluation Q-network updates θ in each training step, while the target Q network fixes θ' when calculates y_t and will update after K steps.

During each training epoch of the D3QN network, the gradient descent algorithm is utilized to minimize the loss function to find the optimal parameters of the predict network, which is further used to evaluate the Q value of each chosen action. We build the loss function of the DDQN algorithm as

$$L(\theta) = \mathbb{E} \left[(y_t - Q(s_t, a_t; \theta))^2 \right] \quad (17)$$

In addition, there is a correlation between the Q value samples obtained by the same approximation method, and directly using them as training data will result in over fitting. In order to break the association between samples and make them independent, we apply experience replay technology to our algorithm. The collected samples in each time step (s_t, a_t, r_t, s_{t+1}) are denoted as experience, and will be stored into experience memory D. Then a mini-batch of samples (s_j, a_j, r_j, s_{j+1}) are randomly selected from D for network weights training. This can improve the training efficiency of the network and obtain better convergence.

The proposed computation offloading and resource allocation algorithm based on D3QN is explained in [Algorithm 1](#). When a M-IoT device generates a compute task to be offloaded, the M-IoT device sends a request to the MEC server. All M-IoT devices and MEC servers need to transmit their own state to the SDN controller, which can perform actions (compute offload decisions, power allocation, and compute resource allocation) and receive instant rewards, as well as store the obtained sample sequences in the experience replay memory. To do this, D3QN randomly selects small batches of samples from the experience replay memory as training samples to train the main network, then calculates losses and gradients of the D3QN main network. Finally, the D3QN main network parameters are updated by stochastic gradient descent method. All parameters of the main network are copied to a target network every K steps.

Algorithm 1. D3QN based computation offloading and resource allocation.

1. Initialize replay buffer D
2. Initialize the evaluation network Q with parameters θ
3. Initialize the target network Q' with parameters $\theta' \leftarrow \theta$
4. for each episode, do
5. Initialize environment, observe state s_t
6. for each step, do
7. Select action a_t based on ε -greedy strategy
8. otherwise select $a_t = \max_a Q(s_t, a; \theta)$
9. Execute action a_t , observe reward r_t and next state s_{t+1}
10. Store transition $\{s_t, a_t, s_{t+1}, r_t\}$ in D
11. Randomly sample mini-batch of transitions from D
12. Compute the target network value y_t :

$$y_t = r_t + \gamma \max_{a'} Q' \left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta); \theta' \right)$$

13. Compute the loss function and perform gradient descent to update θ

$$L(\theta) = \mathbb{E} \left[(y_t - Q(s_t, a_t; \theta))^2 \right]$$

14. Every K steps, reset $\theta' \leftarrow \theta$ **15. end for****16. end for**

4. Simulation results and discussions

In this section, we demonstrate the simulation experimental results of the proposed algorithm to assess its performance. We evaluate and analyze the convergence of the proposed algorithm with different hyperparameters. In addition, we compare the performance of the proposed algorithm with that of other DRL algorithms in term of system cost and delay.

4.1. Simulation parameters setting

This experiment uses PyCharm as the experimental environment and implements D3QN algorithm based on PyTorch framework. The experimental scenario simulates the Marine mobile edge computing system composed of multiple devices carrying MEC servers, such as ships, UAVs, etc., and multiple M-IoT devices. In the experiment, we plan and deploy the edge server ahead of time, while the mobile user moves along a predetermined route for computing offloading and resource allocation experiment simulation. Referring to some related research [20,21,25], and based on communication and computation constraints of the M-IoT scenario. The detailed parameters of the simulation environment are shown in Table 1. The relevant parameters of D3QN algorithm are listed in Table 2.

Table 1. Simulation Parameters.

Parameters	Value
The number of M-IoTs (N)	[3,29]
The number of MECs (M)	5
Bandwidth (B)	1MHz
Noise power (σ^2)	-100 dBm
Transmission power of each MIoT (p_i)	[29,34] dBm
Maximum transmit power (P_{max})	35 dBm
Input data size (B_i)	[1,5] MB
Delay Tolerance (τ_i^{max})	0.5 s
Computation capability of UEs (f_i^l)	1 GHz
Computation capability of MEC (f_j)	[1,5] GHz
Energy coefficient	10^{-27}

Table 2. D3QN Parameters.

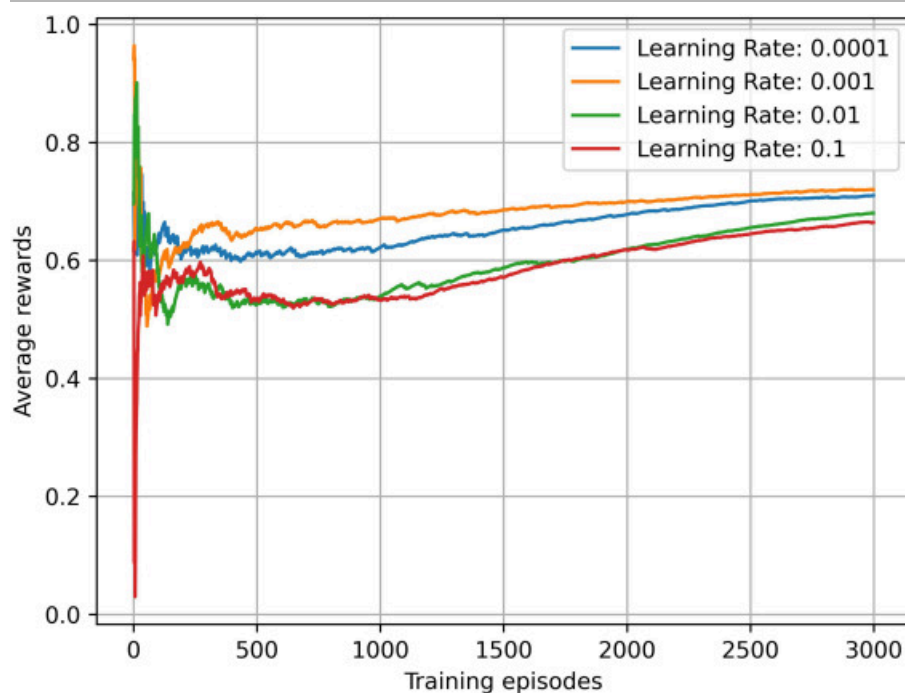
Parameters	Value
The number of layers in the network	3

Parameters	Value
The number of fully connected layers	[512,256,128]
Learning rate (lr)	{0.1,0.01,0.001,0.001}
Discount factor (γ)	{0.9,0.95, 0.99,0.999}
Batch size	64
Replay Period	3000
Activation function	ReLU
ϵ -greedy probability	0.9

Similar to [33,34], to guarantee consistent performance, we set the replay memory size to 1000 and the minibatch size to 64 to reduce sample correlation and improve training stability. The discount factor γ is set to 0.95, which is used to calculate the discount rate of future rewards and balance the short-term and long-term gains. epsilon-greedy ϵ is set to 0.995 to balance exploration and exploitation. The learning rate (lr) is set to 0.001 to prevent convergence too fast or falling into local optimum. The DNN structure comprises two hidden layers with 512 and 256 neurons, respectively. The ReLU function is used for the activation function, and the Adam Optimizer is used to optimize the loss function, and we perform all of the simulations using 3000 episodes.

4.2. Simulation results

The settings of different hyperparameters will have a great impact on the training of neural networks. First of all, when the learning rate is too large, it is easy to have convergence miscalculation, which will affect later performance effect in the later stage and fluctuate to a greater extent. When the learning rate is too small, the slow learning process increases the convergence time. We examined the impact of different learning rates on the convergence performance of the proposed D3QN algorithm, as shown in Fig. 4.

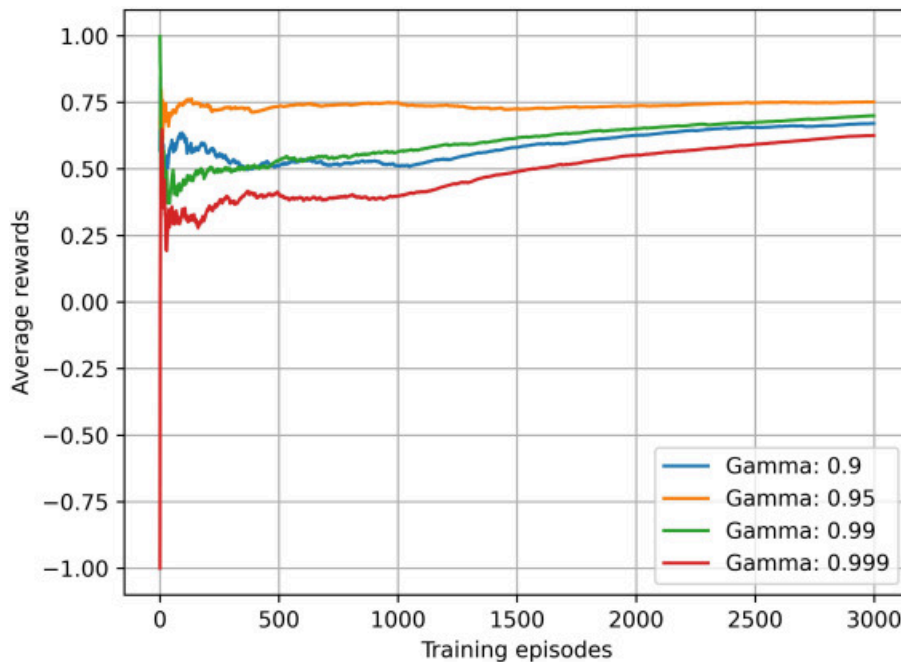


[Download: Download high-res image \(408KB\)](#)

[Download: Download full-size image](#)

Fig. 4. Convergence performance of proposed D3QN algorithm with different learning rates.

Fig. 4 illustrates the convergence performance of the proposed D3QN algorithm under different learning rates. Specifically, when the learning rate is set to 0.1, the fluctuation is large, which is prone to convergence misclassification and affects the performance effect in the later stage. The optimal convergence position is achieved when the learning rate is 0.001. It is noteworthy that due to the stochastic nature of the environment in each episode, the converged reward may still fluctuate. Similarly, Fig. 5 illustrates the convergence performance of the D3QN algorithm under different discount factors. The discount factor is used in reinforcement learning to balance immediate and future rewards, in our application scenario, optimization of long-term behaviors and strategies is more critical than short-term rewards, and with reference to previous research support, we chose $\gamma = 0.99$ to evaluate the effectiveness of long-term strategies. Therefore, in the following simulations, the parameters are fixed to $lr = 0.001$ and $\gamma = 0.99$.

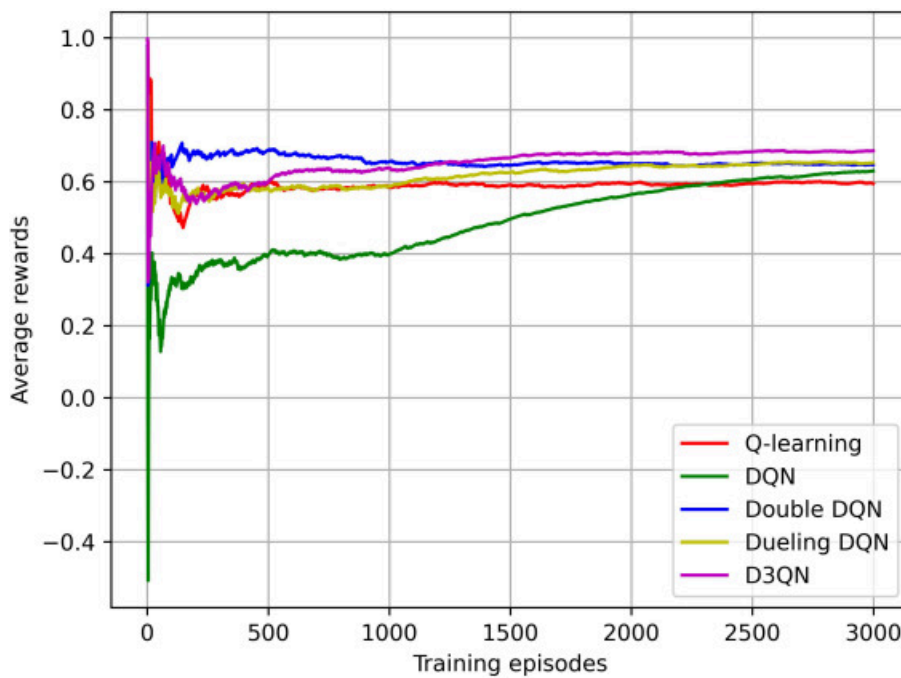


Download: [Download high-res image \(367KB\)](#)

Download: [Download full-size image](#)

Fig. 5. Convergence performance of proposed D3QN algorithm with different GAMMA.

The D3QN algorithm proposed in this paper is based on improving the Double DQN algorithm [35] and the Dueling DQN [36] algorithm. Therefore, in the experimental comparison, these two algorithms are introduced to compare with the D3QN algorithm. Then, to calculate the D3QN algorithm compared with the two improved DQN algorithms for the optimization degree of the basic DQN algorithm, this paper introduces the DQN [37] and Q-Learning algorithm to conduct comparative experiments simultaneously. The experimental result is shown in Fig. 6.



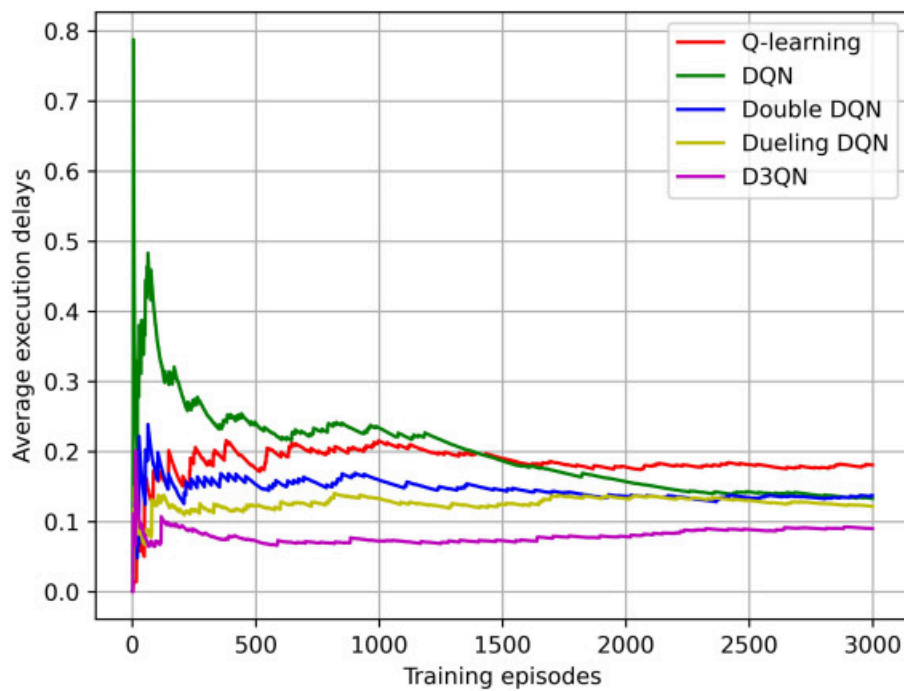
[Download: Download high-res image \(412KB\)](#)

[Download: Download full-size image](#)

Fig. 6. Comparison of average rewards with episodes.

Fig. 6 shows the convergence performance of different DRL algorithms. As can be seen from the figure, during the training process, the reward value will fluctuate slightly, which is the impact of the action selection strategy. Since the Q-learning method requires the establishment of a large-scale state action table, this results in a large amount of calculation and limitations in high-dimensional environments. The Double DQN algorithm introduces another neural network to reduce the impact of errors, and thus outperforms Q-learning in terms of computation efficiency, but may cause the estimation error in the algorithm to become unstable. By introducing a dual network architecture and splitting the value function, D3QN can better solve the overestimation problem and action selection bias problem in the DQN algorithm. This makes D3QN more stable during the learning process and easier to converge to the optimal strategy. In addition, in multiple rounds of simulation experiments, Dueling DQN was found to have weaker stability compared to D3QN.

Fig. 7 compares the change of the average task delay with the number of iterations of five different algorithms. As shown in this figure, Q-learning, DQN, Double DQN and Dueling DQN change dynamically in average delay when processing tasks. In contrast, the proposed D3QN algorithm has a relatively low average delay under the same number of iterations, which can effectively reduce the task offloading processing delay and show better stability, so as to better select the decision for task offloading processing. The D3QN algorithm combines the advantages of the Double DQN and the Dueling DQN, which makes it possible to more accurately evaluate the value of the action during the decision-making process and reduce the uncertainty in the decision-making process.

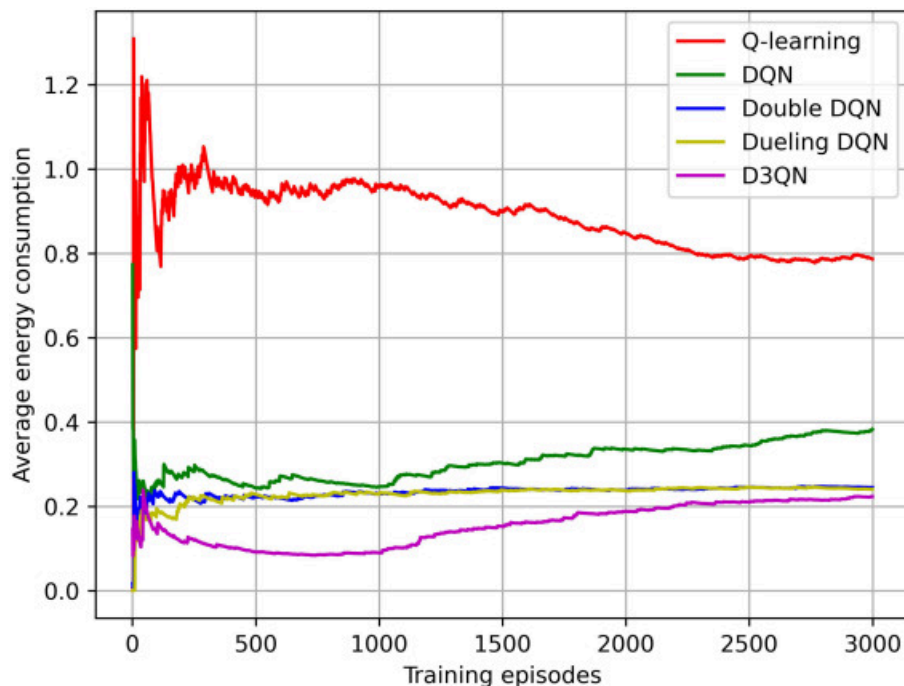


Download: [Download high-res image \(474KB\)](#)

Download: [Download full-size image](#)

Fig. 7. Comparison of average delay with episodes.

Fig. 8 shows the average energy consumption of five different algorithms with the number of iterations. As shown in this figure, the average energy consumption of the system gradually tends to stabilize with the increase of the number of iterations. Whereas Q-learning may be difficult to converge quickly in complex scenario, resulting in a higher average energy consumption and larger delay fluctuations. Compared to the other algorithms, the energy consumption of the proposed D3QN algorithm reaches the lowest, which it shows that the whole network can better utilize the computing resources and improve the energy efficiency of the whole network.

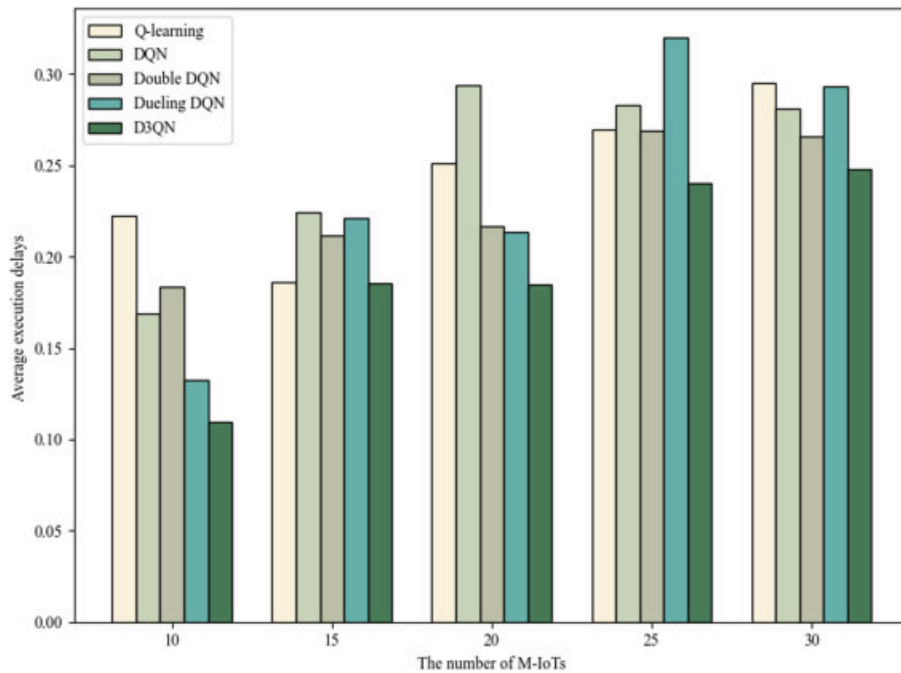


Download: [Download high-res image \(452KB\)](#)

Download: [Download full-size image](#)

Fig. 8. Comparison of average energy consumption with episodes.

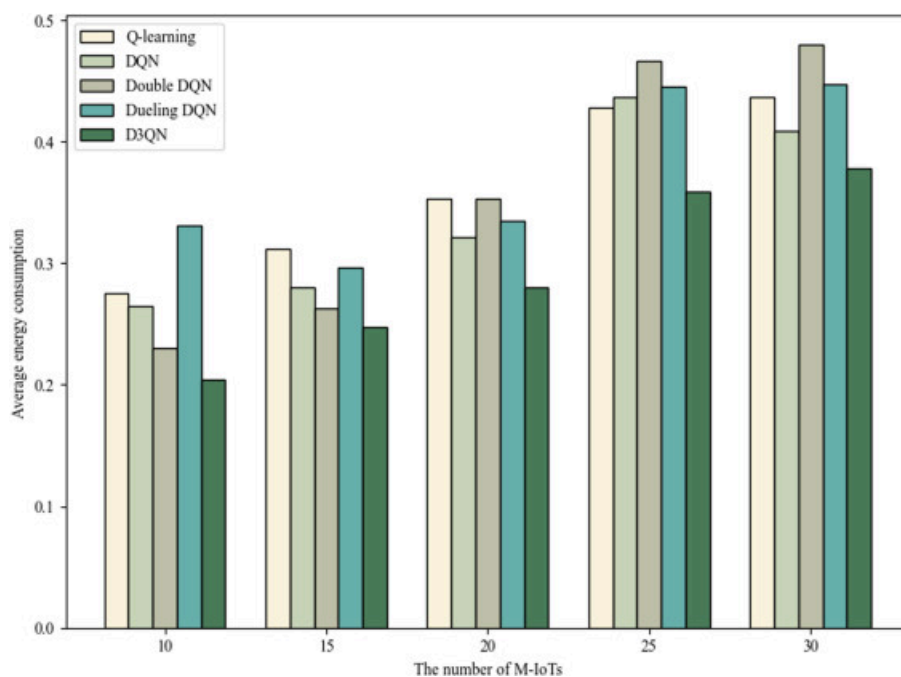
Fig. 9, Fig. 10 show the algorithm's performance in terms of average delay and energy consumption as the number of M-IoTs increases, respectively. With the increase of the number of user devices, the channel interference gradually increases, the transmission rate decreases, and the total number of computing tasks to be processed in each time slot also increases, resulting in the simultaneous increase of the average delay and energy consumption. However, under the same number of user devices, the D3QN algorithm can always achieve the lowest average delay and energy consumption, which indicates that the algorithm has a higher efficiency in the allocation and utilization of computing resources. When the computing tasks of multiple users are processed in parallel in the same system, the algorithm can more intelligently decide the task offloading strategy, so that the computing resources can be allocated and scheduled more reasonably, to effectively improve the quality of service and ensure the stability and reliability of the system.



[Download: Download high-res image \(281KB\)](#)

[Download: Download full-size image](#)

Fig. 9. Comparison of average delay with different numbers of M-IoTs.



[Download: Download high-res image \(278KB\)](#)

[Download: Download full-size image](#)

Fig. 10. Comparison of average energy consumption with different numbers of M-IoTs.

In summary, under the same conditions, we compare the proposed D3QN algorithm with the existing Dueling DQN, Double DQN, DQN and Q-Learning algorithms. From the comparison of performance metrics of average network reward, average delay and average energy consumption, the proposed algorithm significantly outperforms better than other DRL algorithms, and the proposed algorithm has faster convergence speed and stable convergence performance with less convergence fluctuation. This proves that the algorithm can effectively reduce the system task delay and energy consumption in task offloading and resource allocation, reduce the total cost of the system, and thus improve the performance of the whole network system.

5. Conclusion

This paper introduces a software-defined SAGSIN architecture based on mobile edge computing to cope with the growing demand for computationally intensive applications in marine environments. On the basis of the architecture, we design a joint optimization problem for computation offloading and resource allocation to address the limitation of marine resources and the instability of marine channel. To avoid the curse of dimensionality caused by the exponential increase of state–action space, we proposed a D3QN-based method. This algorithm can balance delay and energy consumption and improve offloading efficiency without requiring global network environment information, which is especially suitable for resource-constrained marine environments. The experimental results demonstrate that the proposed D3QN algorithm exhibits excellent convergence performance and system overhead in marine scenarios, significantly reduces system delay and energy consumption, and is able to efficiently meeting users' service requirements.

CRedit authorship contribution statement

Yanli Xu: Writing – review & editing, Supervision, Methodology, Investigation, Funding acquisition. **Qianlian Yu:** Writing – original draft, Visualization, Validation, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

[Special issue articles](#) [Recommended articles](#)

Data availability

No data was used for the research described in the article.

References

- [1] T. Wei, W. Feng, Y. Chen, *et al.*
Hybrid satellite-terrestrial communication networks for the maritime Internet of Things: key technologies, opportunities, and challenges
IEEE Int. Things J., 8 (11) (2021), pp. 8910–8934
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [2] T. Xia, M.M. Wang, J. Zhang, *et al.*
Maritime Internet of things: Challenges and Solutions, 27, IEEE Wireless Communications (2020), pp. 188–196
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [3] Y. Huo, X. Dong, S. Beatty

Cellular communications in ocean waves for maritime internet of Things

IEEE Int. Things J., 7 (10) (2020), pp. 9965-9979

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[4] M. Dai, N. Huang, Y. Wu, *et al.*

Latency minimization oriented hybrid offshore and aerial-based multi-access computation offloading for marine communication networks

IEEE Trans. Commun. (2023)

[Google Scholar ↗](#)

[5] X. Li, W. Feng, Y. Chen, *et al.*

Maritime coverage enhancement using UAVs coordinated with hybrid satellite-terrestrial networks

IEEE Trans. Commun., 68 (4) (2020), pp. 2355-2369

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[6] Y. Liu, L. Jiang, Q. Qi, *et al.*

Online computation offloading for collaborative space/aerial-aided edge computing toward 6G system

IEEE Trans. Veh. Technol. (2023)

[Google Scholar ↗](#)

[7] M. Hosseini, R. Ghazizadeh, H. Farhadi

Game theory-based radio resource allocation in NOMA vehicular communication networks supported by UAV

Phys. Comm., 52 (2022), Article 101681

 [View PDF](#) [View article](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[8] S.S. Hassan, Y.K. Tun, N.H. Tran, *et al.*

Seamless and energy-efficient maritime coverage in coordinated 6G space–Air–Sea non-terrestrial networks

IEEE Int. Things J., 10 (6) (2022), pp. 4749-4769

[Google Scholar ↗](#)

[9] F. Xu, F. Yang, C. Zhao, *et al.*

Deep reinforcement learning based joint edge resource management in maritime network

China Comm., 17 (5) (2020), pp. 211-222

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[10] N. Zhang, S. Zhang, P. Yang, *et al.*

Software defined space-air-ground integrated vehicular networks: challenges and solutions

IEEE Commun. Mag., 55 (7) (2017), pp. 101-109

[Google Scholar ↗](#)

[11] H. Wu, J. Chen, C. Zhou, *et al.*

Resource Management in Space-Air-Ground Integrated Vehicular Networks: SDN Control and AI Algorithm Design, 27, IEEE Wireless Communications (2020), pp. 52-60

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[12] Z. Liao, C. Chen, Y. Ju, *et al.*

Multi-controller deployment in SDN-enabled 6G space–air–ground integrated network

Remote Sens (Basel), 14 (5) (2022), p. 1076

[Crossref ↗](#) [Google Scholar ↗](#)

- [13] Gakpo G.K., Su X., Choi C. Moving intelligence of mobile edge computing to maritime network[C]//Proceedings of the Conference on Research in Adaptive and Convergent Systems. 2019: 189–193.
[Google Scholar ↗](#)
- [14] J. Zeng, J. Sun, B. Wu, *et al.*
Mobile edge communications, computing, and caching (MEC3) technology in the maritime communication network
China Comm., 17 (5) (2020), pp. 223-234
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [15] A. Xiao, H. Chen, S. Wu, *et al.*
Voyage-Based Computation Offloading For Secure Maritime Edge Networks[C]//2020 IEEE Globecom Workshops (GC Wkshps
IEEE (2020), pp. 1-6
[Crossref ↗](#) [Google Scholar ↗](#)
- [16] G. Yue, C. Huang, X. Xiong
A task offloading scheme in Maritime Edge computing Network
J. Comm. Inf. Networks, 8 (2) (2023), pp. 171-186
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [17] T. Yang, H. Feng, C. Yang, *et al.*
Multivessel computation offloading in maritime mobile edge computing network
IEEE Int. Things J., 6 (3) (2018), pp. 4063-4073
[Google Scholar ↗](#)
- [18] T. Yang, S. Gao, J. Li, *et al.*
Multi-armed bandits learning for task offloading in maritime edge intelligence networks
IEEE Trans. Veh. Technol., 71 (4) (2022), pp. 4212-4224
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [19] D. Wang, T. He, Y. Lou, *et al.*
Double-edge computation offloading for Secure Integrated Space-air-aqua networks
IEEE Int. Things J. (2023)
[Google Scholar ↗](#)
- [20] T. Yang, H. Feng, S. Gao, *et al.*
Two-stage offloading optimization for energy–latency tradeoff with mobile edge computing in maritime Internet of Things
IEEE Int. Things J., 7 (7) (2019), pp. 5954-5963
[Google Scholar ↗](#)
- [21] L. Zhang, N. Ansari
Latency-aware IoT service provisioning in UAV-aided mobile-edge computing networks
IEEE Int. Things J., 7 (10) (2020), pp. 10573-10580
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [22] Z. Yang, C. Pan, K. Wang, *et al.*
Energy efficient resource allocation in UAV-enabled mobile edge computing networks
IEEE Trans. Wireless Commun., 18 (9) (2019), pp. 4576-4589
[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)
- [23] Z. Wang, B. Lin, Q. Ye, *et al.*

Joint computation offloading and resource allocation for Maritime MEC with energy harvesting
IEEE Int. Things J. (2024)

[Google Scholar ↗](#)

- [24] You J., Jia Z., Dong C., et al. Computation Offloading for Uncertain Marine Tasks by Cooperation of UAVs and Vessels. arXiv preprint arXiv:2302.06055, 2023.

[Google Scholar ↗](#)

- [25] Z. Wang, B. Lin, L. Sun, et al.
Intelligent task offloading for 6G-enabled maritime IoT based on reinforcement learning 2021 International conference on Security, Pattern Analysis, and Cybernetics (SPAC)
IEEE (2021), pp. 566-570

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

- [26] N. Zhao, Z. Ye, Y. Pei, et al.
Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing
IEEE Trans. Wireless Commun., 21 (9) (2022), pp. 6949-6960

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

- [27] L.P. Qian, H. Zhang, Q. Wang, et al.
Joint multi-domain resource allocation and trajectory optimization in UAV-assisted Maritime IoT networks
IEEE Int. Things J. (2022)

[Google Scholar ↗](#)

- [28] Y. Xu
Gradient-free scheduling of fog computation for marine data feedback
IEEE Int. Things J., 8 (7) (2020), pp. 5657-5668

[Google Scholar ↗](#)

- [29] J. Wang, H. Zhou, Y. Li, et al.
Wireless channel models for maritime communications
IEEE Access, 6 (2018), pp. 68070-68088

[Crossref ↗](#) [Google Scholar ↗](#)

- [30] W. Chen, G. Shen, K. Chi, et al.
DRL based partial offloading for maximizing sum computation rate of FDMA-based wireless powered mobile edge computing
Comput. Networks, 214 (2022), Article 109158

 [View PDF](#) [View article](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

- [31] Dulac-Arnold G., Mankowitz D., Hester T. Challenges of real-world reinforcement learning. arXiv preprint arXiv:1904.12901, 2019.

[Google Scholar ↗](#)

- [32] H. Hu, D. Wu, F. Zhou, et al.
Intelligent resource allocation for edge-cloud collaborative networks: a hybrid DDPG-D3QN approach
IEEE Trans. Veh. Technol., 72 (8) (2023), pp. 10696-10709

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

- [33] F. Jiang, Y. Li, C. Sun, et al.

Dueling double deep q-network based computation offloading and resource allocation scheme for internet of vehicles

2023 IEEE Wireless Communications and Networking Conference (WCNC), IEEE (2023), pp. 1-6

[Google Scholar ↗](#)

[34] W. Cheng, X. Liu, X. Wang, *et al.*

Task offloading and resource allocation for industrial Internet of Things: a double-dueling deep Q-network approach

IEEE Access, 10 (2022), pp. 103111-103120

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[35] Van Hasselt H., Guez A., Silver D. Deep reinforcement learning with double q-learning[C]//Proceedings of the AAAI conference on artificial intelligence. 2016, 30(1).

[Google Scholar ↗](#)

[36] Z. Wang, T. Schaul, M. Hessel, *et al.*

Dueling network architectures for deep reinforcement learning

International Conference on Machine Learning, PMLR (2016), pp. 1995-2003

[View in Scopus ↗](#) [Google Scholar ↗](#)

[37] Roderick M., MacGlashan J., Tellex S. Implementing the deep q-network. arXiv preprint arXiv:1711.07478, 2017.

[Google Scholar ↗](#)

Cited by (3)

A privacy-aware and sustainable joint optimization for resource-constrained internet of things using deep reinforcement learning

2026, Internet of Things the Netherlands

[Show abstract](#) ✓

From 6G to SeaX-G: Integrated 6G TN/NTN for AI-Assisted Maritime Communications—Architecture, Enablers, and Optimization Problems ↗

2025, Journal of Marine Science and Engineering

Collaborative DNN Inference in Maritime Edge Intelligence Networks with Group Neural Multi-Armed Bandits ↗

2025, Intelligent and Converged Networks



Yanli Xu received the Ph.D. degree from the National Key Laboratory of Mobile Communications, Southeast University, Nanjing, China, in 2012. From 2012 to 2015, she worked as a research scientist in Bell labs of Alcatel-Lucent, China, with a focus on the standardization of 5 G cellular networks. Now she is an associate professor of Shanghai Maritime University. She is an associate editor of IEEE Access, and serves as a TPC member or reviewer for many conferences and journals, such as IEEE JSAC, TCOM, TWC, TSP, IET Com, Globecom, etc. Her research interests are in 6 G, maritime communications and IoT, with a current focus on edge computation.



Qianlian Yu received the B.S. degree in from the electronic information engineering from Sanjiang University in 2021. She is currently pursuing the M.S. degree in the school of information and communication engineering at Shanghai Maritime University. Her research interests include maritime communication, mobile edge computing and deep reinforcement learning.

[View Abstract](#)

© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

